

React Hooks

A Complete Reference Guide with Code Examples

This guide covers every built-in React hook — state hooks, context hooks, ref hooks, effect hooks, performance hooks, and the newer React 18/19 hooks — along with its purpose, signature, parameters, return values, and a minimal, runnable code example.

1. State Hooks — `useState`, `useReducer`
2. Context Hook — `useContext`
3. Ref Hooks — `useRef`, `useImperativeHandle`
4. Effect Hooks — `useEffect`, `useLayoutEffect`, `useInsertionEffect`
5. Performance Hooks — `useMemo`, `useCallback`, `useTransition`, `useDeferredValue`
6. Other Hooks — `useId`, `useSyncExternalStore`, `useDebugValue`, `use()`
7. Custom Hooks — building your own

1. State Hooks

State hooks let a component “remember” information between renders.

useState local component state

Purpose: Adds a single piece of state to a function component and returns a setter to update it.

Signature: `const [state, setState] = useState(initialState)`

Parameters

- **initialState** — the value used on the first render, or a function that returns it (lazy init)

Returns

- **state** — the current value
- **setState** — function to update the value and trigger a re-render

Notes

- Updates are batched and may be asynchronous.
- Pass a function to `setState` to safely update based on the previous value.
- Calling the setter with the same value (`Object.is`) bails out of re-rendering.

Example

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(c => c + 1)}>
      Count: {count}
    </button>
  );
}
```

useReducer state via reducer function

Purpose: Manages more complex state logic using a reducer function, similar to Redux.

Signature: `const [state, dispatch] = useReducer(reducer, initialArg, init?)`

Parameters

- **reducer** — `(state, action) => newState`
- **initialArg** — the initial state value
- **init** — optional function to lazily compute the initial state

Returns

- **state** — the current state
- **dispatch** — function to send an action to the reducer

Notes

- Preferred over `useState` when the next state depends on the previous one in complex ways.
- Great for state machines and forms with many related fields.

Example

```
import { useReducer } from 'react';

function reducer(state, action) {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: throw new Error('Unknown action');
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      <span>{state.count}</span>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
    </>
  );
}
```

2. Context Hook

useContext read shared/global data

Purpose: Reads and subscribes to a React context, avoiding prop-drilling.

Signature: `const value = useContext(SomeContext)`

Parameters

- **SomeContext** — the context object created with `createContext()`

Returns

The current context value provided by the nearest matching `<Provider>` above in the tree.

Notes

- Component re-renders automatically whenever the context value changes.
- If no Provider exists above, the default value passed to `createContext()` is used.

Example

```
import { createContext, useContext } from 'react';

const ThemeContext = createContext('light');

function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>I am {theme}</button>;
}

function App() {
  return (
    <ThemeContext.Provider value='dark'>
      <ThemedButton />
    </ThemeContext.Provider>
  );
}
```

3. Ref Hooks

useRef mutable value / DOM access

Purpose: Creates a mutable ref object that persists for the component's lifetime without causing re-renders when it changes.

Signature: `const ref = useRef(initialValue)`

Parameters

- **initialValue** — the value to set `ref.current` to initially

Returns

- **ref** — an object of the shape `{ current: initialValue }`

Notes

- Mutating `ref.current` does NOT trigger a re-render.
- Commonly used to reference DOM nodes or store any mutable value across renders.

Example

```
import { useRef } from 'react';

function TextInputWithFocusButton() {
  const inputRef = useRef(null);

  const focusInput = () => inputRef.current.focus();

  return (
    <>
      <input ref={inputRef} type='text' />
      <button onClick={focusInput}>Focus input</button>
    </>
  );
}
```

useImperativeHandle customize exposed ref

Purpose: Customizes the instance value that is exposed to parent components when using ref with forwardRef.

Signature: useImperativeHandle(ref, createHandle, deps?)

Parameters

- **ref** — the forwarded ref from the parent
- **createHandle** — function returning the object to expose
- **deps** — dependency array controlling when the handle is recreated

Returns

Nothing (undefined) — it configures what the parent's ref.current will contain.

Notes

- Used together with forwardRef.
- Lets you expose an imperative API (e.g. focus(), scrollTo()) instead of the raw DOM node.

Example

```
import { forwardRef, useRef, useImperativeHandle } from 'react';

const FancyInput = forwardRef((props, ref) => {
  const inputRef = useRef(null);
  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(),
  }));
  return <input ref={inputRef} />;
});

function Parent() {
  const ref = useRef(null);
  return (
    <>
      <FancyInput ref={ref} />
      <button onClick={() => ref.current.focus()}>Focus</button>
    </>
  );
}
```

4. Effect Hooks

useEffect sync with external systems

Purpose: Runs side effects (data fetching, subscriptions, manual DOM changes) after render, and can clean up before the next run or unmount.

Signature: `useEffect(setupFn, deps?)`

Parameters

- **setupFn** — function that runs after commit; may return a cleanup function
- **deps** — array of values the effect depends on. Omit to run every render; `[]` to run once

Returns

Nothing (undefined).

Notes

- Runs asynchronously, after the browser has painted.
- The cleanup function runs before the next effect and on unmount.

Example

```
import { useEffect, useState } from 'react';

function ChatRoom({ roomId }) {
  const [messages, setMessages] = useState([]);

  useEffect(() => {
    const connection = createConnection(roomId);
    connection.on('message', m => setMessages(msgs => [...msgs, m]));
    connection.connect();
    return () => connection.disconnect(); // cleanup
  }, [roomId]);

  return <ul>{messages.map(m => <li key={m.id}>{m.text}</li>)}</ul>;
}
```

useLayoutEffect synchronous DOM measurement

Purpose: Like `useEffect`, but fires synchronously after DOM mutations and before the browser paints.

Signature: `useLayoutEffect(setupFn, deps?)`

Parameters

- Same as `useEffect`: **setupFn** and optional **deps** array

Returns

Nothing (undefined).

Notes

- Blocks the browser paint, so use sparingly — only when you must measure/mutate the DOM before the user sees it.
- Prefer `useEffect` unless you see visual flicker.

Example

```
import { useLayoutEffect, useRef, useState } from 'react';

function Tooltip() {
  const ref = useRef(null);
  const [height, setHeight] = useState(0);

  useLayoutEffect(() => {
    const { height } = ref.current.getBoundingClientRect();
    setHeight(height); // measured before paint, no flicker
  }, []);

  return <div ref={ref}>Tooltip is {height}px tall</div>;
}
```


useInsertionEffect inject styles before layout

Purpose: Fires before any DOM mutations are read by `useLayoutEffect`; designed for CSS-in-JS libraries to inject `<style>` tags.

Signature: `useInsertionEffect(setupFn, deps?)`

Parameters

- Same signature as `useEffect` / `useLayoutEffect`

Returns

Nothing (undefined).

Notes

- Very rarely needed in application code — mainly for CSS-in-JS library authors.
- Refs are not yet attached and you should not update state here.

Example

```
import { useInsertionEffect } from 'react';

function useCSS(rule) {
  useInsertionEffect(() => {
    const style = document.createElement('style');
    style.textContent = rule;
    document.head.appendChild(style);
    return () => document.head.removeChild(style);
  }, [rule]);
}
```

5. Performance Hooks

useMemo memoize a computed value

Purpose: Caches the result of an expensive calculation between re-renders until its dependencies change.

Signature: `const value = useMemo(calcFn, deps)`

Parameters

- **calcFn** — function that computes and returns the value
- **deps** — dependency array

Returns

The memoized value from the last time `calcFn` ran (or a fresh computation if `deps` changed).

Notes

- Only use for genuinely expensive computations — it has its own overhead.
- Does not guarantee the value is never recomputed (React may discard the cache under memory pressure).

Example

```
import { useMemo } from 'react';

function ProductList({ products, query }) {
  const filtered = useMemo(
    () => products.filter(p => p.name.includes(query)),
    [products, query]
  );
  return <ul>{filtered.map(p => <li key={p.id}>{p.name}</li>)}</ul>;
}
```

useCallback memoize a function reference

Purpose: Returns a memoized version of a callback that only changes if its dependencies change.

Signature: `const fn = useCallback(callback, deps)`

Parameters

- **callback** — the function to memoize
- **deps** — dependency array

Returns

The same function instance across renders as long as deps are unchanged.

Notes

- Useful when passing callbacks to memoized child components (`React.memo`) to avoid unnecessary re-renders.
- `useCallback(fn, deps)` is equivalent to `useMemo(() => fn, deps)`.

Example

```
import { useCallback, useState } from 'react';

function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    setCount(c => c + 1);
  }, []); // never changes identity

  return <MemoizedChild onClick={handleClick} />;
}
```

useTransition mark updates as non-urgent

Purpose: Lets you mark a state update as a low-priority 'transition', keeping the UI responsive while it renders.

Signature: `const [isPending, startTransition] = useTransition()`

Returns

- **isPending** — boolean, true while the transition is running
- **startTransition** — function to wrap a non-urgent state update

Notes

- Great for expensive UI updates like filtering large lists while typing stays responsive.
- Transitions can be interrupted by more urgent updates.

Example

```
import { useTransition, useState } from 'react';

function SearchPage() {
  const [isPending, startTransition] = useTransition();
  const [results, setResults] = useState([]);

  function handleChange(e) {
    const query = e.target.value;
    startTransition(() => {
      setResults(search(query)); // low priority
    });
  }

  return (
    <>
      <input onChange={handleChange} />
      {isPending ? <Spinner /> : <ResultsList items={results} />}
    </>
  );
}
```

useDeferredValue defer re-rendering a value

Purpose: Returns a deferred copy of a value that 'lags behind' during urgent updates, letting React skip stale renders.

Signature: `const deferredValue = useDeferredValue(value, initialValue?)`

Parameters

- **value** — the value to defer
- **initialValue** — optional value to use on first render

Returns

The current value, or a previous (stale) version while a newer render is being prepared.

Notes

- Useful for keeping typing responsive while a derived, expensive view lags slightly behind.
- Similar goal to `useTransition` but used when you don't control the state setter.

Example

```
import { useDeferredValue, useState } from 'react';

function SearchResults({ query }) {
  const deferredQuery = useDeferredValue(query);
  const results = useMemo(() => search(deferredQuery), [deferredQuery]);
  return <ResultsList items={results} />;
}
```

6. Other Hooks

useId unique, stable IDs

Purpose: Generates a unique, stable ID string that is consistent between server and client renders, useful for accessibility attributes.

Signature: `const id = useId()`

Returns

A unique ID string (e.g. 'r1:').

Notes

- Do NOT use it as a React key or as a database/list ID — it is only for markup relationships like `htmlFor/id`.
- Prevents hydration mismatches in server-rendered apps.

Example

```
import { useId } from 'react';

function PasswordField() {
  const id = useId();
  return (
    <>
      <label htmlFor={id}>Password</label>
      <input id={id} type='password' />
    </>
  );
}
```

useSyncExternalStore subscribe to external stores

Purpose: Subscribes a component to an external (non-React) data store in a way that is safe for concurrent rendering.

Signature: `const state = useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?)`

Parameters

- **subscribe** — function that registers a callback and returns an unsubscribe function
- **getSnapshot** — function returning the current store value
- **getServerSnapshot** — optional snapshot function used during server rendering

Returns

The current value of the store.

Notes

- The recommended way to integrate third-party state libraries (e.g. Redux-like stores) with React 18+.
- Used internally by many state-management libraries.

Example

```
import { useSyncExternalStore } from 'react';

function useOnlineStatus() {
  return useSyncExternalStore(
    (callback) => {
      window.addEventListener('online', callback);
      window.addEventListener('offline', callback);
      return () => {
        window.removeEventListener('online', callback);
        window.removeEventListener('offline', callback);
      };
    },
    () => navigator.onLine
  );
}
```

useDebugValue label custom hooks in DevTools

Purpose: Displays a label for custom hooks in React DevTools; has no effect on the rendered output.

Signature: useDebugValue(value, formatFn?)

Parameters

- **value** — the value to display
- **formatFn** — optional function to format the value lazily

Returns

Nothing (undefined).

Notes

- Only useful inside custom hooks, not application components.
- formatFn avoids the formatting cost unless DevTools is actually inspecting the hook.

Example

```
import { useDebugValue, useState } from 'react';

function useOnlineStatus() {
  const [isOnline, setIsOnline] = useState(true);
  useDebugValue(isOnline ? 'Online' : 'Offline');
  return isOnline;
}
```


use read a promise or context (React 19)

Purpose: Reads the value of a resource like a Promise or Context. Unlike other hooks, it can be called conditionally and inside loops.

Signature: `const value = use(resource)`

Parameters

- **resource** — a Promise or a Context object

Returns

The resolved value of the promise, or the current context value.

Notes

- When passed a Promise, the component suspends until it resolves (used with `<Suspense>`).
- Introduced in React 19 as a more flexible alternative to `useContext` for conditional reads.

Example

```
import { use, Suspense } from 'react';

function Comments({ commentsPromise }) {
  const comments = use(commentsPromise); // suspends until resolved
  return <ul>{comments.map(c => <li key={c.id}>{c.text}</li>)}</ul>;
}

function Page({ commentsPromise }) {
  return (
    <Suspense fallback={<Spinner />}>
      <Comments commentsPromise={commentsPromise} />
    </Suspense>
  );
}
```

7. Custom Hooks

A custom hook is simply a JavaScript function whose name starts with use and that calls other hooks inside it. Custom hooks let you extract and reuse stateful logic between components without changing the component hierarchy.

Example: useLocalStorage

```
import { useState, useEffect } from 'react';

function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved !== null ? JSON.parse(saved) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Usage:
function Settings() {
  const [theme, setTheme] = useLocalStorage('theme', 'light');
  return (
    <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
      Current theme: {theme}
    </button>
  );
}
```

Rules of Hooks

- Only call hooks at the top level — never inside loops, conditions, or nested functions.
- Only call hooks from React function components or from other custom hooks.
- Hook call order must be identical on every render, since React tracks hooks by call order.